Tecnológico de Monterrey

ITESM Campus Querétaro

TC1038 – Fundamentos de Programación

Proyecto Integrador – Avance 3

Listas, listas de listas y cadenas

Profesor: Dr. Leonardo Ledesma Domínguez

Objetivo de la actividad

En este tercer avance del Proyecto Integrador deberán continuar el desarrollo del programa construido durante el **Avance 2**, incorporando nuevas funcionalidades mediante los temas estudiados recientemente en el curso.
El objetivo de esta entrega es que el proyecto evolucione de un esqueleto funcional hacia una versión con **mayor funcionalidad**, utilizando correctamente **listas, listas de listas, cadenas y funciones adicionales de Python**.
El programa deberá conservar y mejorar los elementos desarrollados previamente: funciones, parámetros, valores de retorno, estructuras condicionales, `match`, ciclos y, cuando corresponda, las librerías `math` y `random`.

1. Incorporación de los nuevos temas

Deberán revisar el código desarrollado en el **Avance 2** e identificar en qué partes del proyecto pueden incorporar los nuevos conocimientos adquiridos.
El programa deberá hacer uso, cuando sean pertinentes para la solución, de los siguientes elementos:

- **Listas** para almacenar, consultar y modificar conjuntos de datos.
- Recorrido y procesamiento de elementos contenidos en listas.
- **Listas de listas** para representar información que requiera una estructura de dos dimensiones, cuando las características del proyecto lo permitan.
- **Cadenas (`str`)** y operaciones para su procesamiento.
- Métodos y operaciones sobre cadenas vistos en clase.
- Integración de estas estructuras con ciclos, condicionales y funciones propias.

No es suficiente declarar una lista, una cadena o una lista de listas. Estas estructuras deberán tener una función real dentro de la solución propuesta.
Si alguna de estas estructuras no resulta pertinente para las características particulares del proyecto, deberá **justificarse mediante un comentario dentro del código**.

2. Uso de funciones y herramientas adicionales de Python

Además de las estructuras anteriores, deberán incorporar **al menos dos funciones, métodos o herramientas adicionales de Python** estudiadas en clase.
Algunos ejemplos son:

- `enumerate()`
- `shuffle()`
- `split()`
- `join()`
- `lower()` / `upper()`
- `append()`
- `remove()`
- `pop()`
- Otras herramientas estudiadas durante el curso.

Las herramientas seleccionadas deberán utilizarse con un **propósito lógico y funcional dentro del proyecto**. No será suficiente agregar instrucciones aisladas únicamente para cumplir con este requisito.
Cuando utilicen `shuffle()`, recuerden que pertenece a la librería `random` y deberán realizar la importación correspondiente.

3. Recuperación de programas desarrollados en clase y tareas

Antes de realizar este avance, deberán **revisar los programas y ejercicios desarrollados previamente en clase y en las tareas**.
En particular, revisen ejemplos como:

- **Memorama**, como referencia para el manejo de listas, listas de listas, posiciones y actualización de información.
- **Ahorcado**, como referencia para el procesamiento de cadenas, recorridos, comparaciones y actualización progresiva de información.
- **Cajero Automático**, como referencia para la integración de funciones, menús, ciclos, condicionales, manejo de información y actualización de valores.
- Ejercicios de manejo y recorrido de listas.
- Ejercicios de cadenas.
- Ejercicios con listas de listas.
- Ejemplos de `enumerate()`, `shuffle()` y otras funciones estudiadas.

El propósito es que identifiquen **estructuras, algoritmos y estrategias de programación que ya conocen** y determinen cuáles pueden adaptar a las necesidades de su proyecto.
No se espera que copien directamente estos programas, sino que recuperen los conocimientos y técnicas utilizadas para construir su propia solución.

4. Evolución respecto al Avance 2

El Avance 3 deberá representar una **evolución real del programa presentado en el Avance 2**. Las funciones que anteriormente fueron declaradas únicamente mediante mensajes deberán revisarse para determinar cuáles **ya pueden ser implementadas con los conocimientos adquiridos hasta este momento del curso**.

Ejemplo de una función que anteriormente podía estar pendiente:

```
def generar_reporte(datos):
    # Pendiente: generar el reporte final
    # utilizando los datos procesados.
    print("Aqui se generara el reporte final.")
```

Aquellas funciones que ahora puedan desarrollarse utilizando listas, listas de listas, cadenas u otras herramientas estudiadas deberán ser implementadas.
Si alguna funcionalidad continúa dependiendo de contenidos que todavía no se han estudiado, podrá mantenerse pendiente, indicando mediante comentarios:

- Qué deberá realizar la función.
- Por qué todavía no se implementa.
- Qué información recibirá o procesará posteriormente.

El objetivo es observar un **desarrollo progresivo del proyecto** conforme se incorporan nuevos conocimientos de programación.

5. Restricción sobre Inteligencia Artificial Generativa

Para este avance no está permitido utilizar herramientas de Inteligencia Artificial Generativa para generar, completar, corregir o modificar el código del proyecto.
El código entregado deberá corresponder al trabajo de los estudiantes y a los conocimientos adquiridos durante el curso.
En caso de que se **identifique el uso de Inteligencia Artificial Generativa en la elaboración del código**, se podrá aplicar una **penalización en la evaluación y una amonestación**, de acuerdo con los **lineamientos de integridad académica y uso de Inteligencia Artificial establecidos y comunicados previamente al inicio del curso**.

6. Organización y documentación del código

El código deberá continuar organizado mediante **funciones con responsabilidades claramente identificables**.
Deberán mantener comentarios descriptivos que permitan comprender:

- El propósito de las principales funciones.
- El uso de las listas y listas de listas.
- La forma en que se procesan las cadenas.
- Las secciones principales del programa.
- Las nuevas funcionalidades incorporadas respecto al Avance 2.
- Las partes que todavía se encuentren pendientes de implementar.

Los comentarios deberán explicar el **propósito o lógica de una sección**, evitando simplemente repetir lo que indica cada instrucción.
Ejemplo:

```
# Recorre la lista de participantes para mostrar
# su posicion y nombre dentro del sistema.
for posicion, nombre in enumerate(participantes):
    print(posicion + 1, nombre)
```

En este caso, el comentario explica **para qué se realiza el recorrido**, en lugar de limitarse a indicar que se está utilizando un ciclo.
El programa deberá mantenerse **ordenado, legible y dividido en funciones**, evitando concentrar toda la lógica dentro de `main()`.

7. Revisión individual del código

Como parte del proceso de evaluación, **la revisora podrá solicitar una reunión individual con cualquier integrante del equipo cuando existan dudas sobre determinadas secciones del código, su funcionamiento o su proceso de desarrollo**.
Durante esta sesión se podrá solicitar al estudiante:

- Explicar la lógica de determinadas secciones del programa.
- Explicar el funcionamiento de alguna función.
- Explicar el uso de listas, listas de listas o cadenas dentro de la solución.
- Explicar las decisiones tomadas para resolver alguna parte del proyecto.
- Responder preguntas relacionadas con la implementación presentada.

Esta revisión tiene como propósito **verificar la comprensión y autoría del código entregado**, así como aclarar posibles dudas relacionadas con el uso no autorizado de herramientas de Inteligencia Artificial Generativa.
La sesión podrá ser **grabada exclusivamente con fines académicos y de seguimiento de la evaluación**, conforme a los lineamientos institucionales aplicables.
La solicitud de una reunión de revisión no implica por sí misma que se haya determinado una falta académica.

8. Entregables

Para completar el Avance 3 deberán entregar los siguientes elementos:

1. **Archivo** `.py` con la nueva versión funcional del proyecto.
2. El programa deberá conservar las funcionalidades desarrolladas en avances anteriores e incorporar las correspondientes a esta entrega.
3. El código deberá contener los **comentarios descriptivos** que faciliten su revisión.
4. Deberán actualizar el proyecto en el **repositorio de GitHub utilizado en el Avance 2**.
5. En la entrega deberán proporcionar el **enlace actualizado al repositorio de GitHub**.

El archivo `.py` entregado y la versión disponible en GitHub deberán corresponder a la **misma versión del proyecto**.

Criterios de evaluación

La evaluación se centrará en la **implementación y aplicación correcta de los conocimientos estudiados hasta este momento del curso**.
Se considerará especialmente:

- Implementación funcional de **listas**.
- Recorrido, consulta y modificación adecuada de listas.
- Implementación de **listas de listas**, cuando sean pertinentes para el proyecto.
- Manejo y procesamiento de **cadenas**.
- Uso pertinente de **al menos dos funciones, métodos o herramientas adicionales de Python**.
- Integración de los nuevos contenidos con funciones, parámetros, `return`, condicionales, `match` y ciclos estudiados previamente.
- Aplicación pertinente de `math` y/o `random`, cuando las características del proyecto lo requieran.
- Evolución real del programa respecto al **Avance 2**.
- Implementación de funciones que anteriormente se encontraban pendientes y que ahora pueden desarrollarse con los conocimientos adquiridos.
- Coherencia del flujo general del programa.
- Organización y modularidad del código.
- Claridad y calidad de los comentarios.
- Legibilidad general del programa.

No se evaluará únicamente la cantidad de código agregado. Se evaluará que los nuevos elementos tengan una aplicación lógica y funcional dentro del proyecto y que representen una evolución respecto al avance anterior.
El propósito del Avance 3 es demostrar que el equipo puede **integrar progresivamente los conocimientos adquiridos en el curso para construir una solución de programación cada vez más completa**.